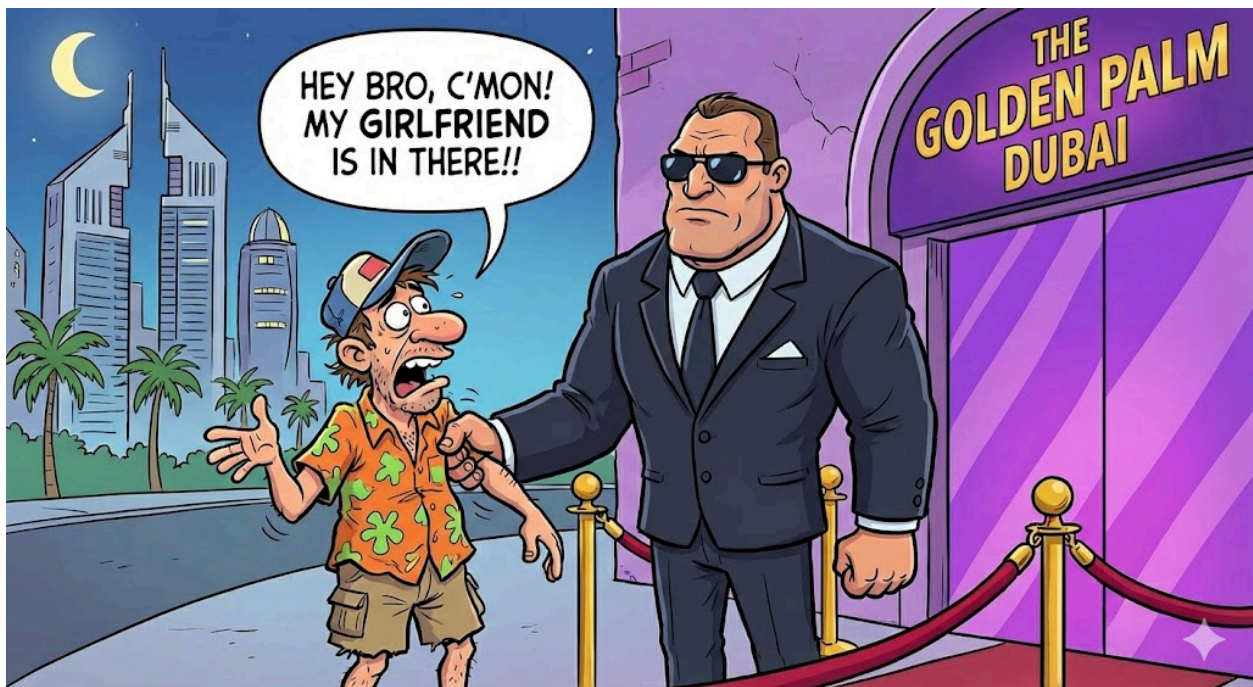


Coding Challenge: Project Overview

Today, we are building **The Doorman Game**, a conversational social engineering simulator. The core loop involves a user attempting to gain entry to an exclusive Dubai-based dance club by persuading an AI-driven Doorman.

Instead of simple keyword matching, the game uses a Dual-Agent Architecture: one agent handles the persona and dialogue of the Doorman, while a second "hidden" agent acts as the Judge, the arbiter of progress. The Judge will determine how persuasive the user is at getting in, and keep track of a hidden progress meter, moving progress up or down depending on the user's messages, the Doorman, and his own backstory.

In the following sections, I'll outline the suggested approach for the project, and then summarize the end goal of the project today.



1. The Core Architecture

The system could operate on a feedback loop involving two distinct LLM instances and a centralized state manager. Here is a starting point you can use as a guide post for your design (*please note: you may deviate from the design below if you believe there is a better approach to accomplish the goals outlined in Section 5*):

Agent A: The Doorman (The Persona)

- **Role:** The front-facing interface.
- **Input:** User message + Personality/Backstory + Full conversation history + Instructions.
- **Output:** Conversational reply to the user.

- **Persona:** Must have a rich, hidden backstory (e.g., "Arthur, a disgruntled former philosophy professor who hates entitlement but has a soft spot for genuine curiosity").
- **Goal:** Maintain character and react naturally to the user's tone and arguments.

Agent B: The Judge (The Logic)

- **Role:** The backend "Game Master."
- **Input:** User's latest reply + Full conversation history + The Doorman's secret principles/personality/backstory + Instructions on how to perform role.
- **Output:** A JSON object containing a brief reasoning analysis of the user's latest reply and then assigning it a numerical "Influence Score" between -20 and +20.
- **Logic:** The Judge evaluates how well the user's input aligns with the Doorman's hidden triggers, backstory, and principles. If the Doorman values "humility," an arrogant bribe attempt might result in a score of -10, whereas a humble request might result in +15.

The State Manager (The Progress Bar)

- Tracks the cumulative **Influence Meter**.
- **Win Condition:** When the cumulative score is over 100, the system triggers a "Grant Access" flag.
 - The State Manager then injects a system instruction to the Doorman: *"The user has convinced you. Let them in now."*
 - The success message should appear organic in the conversation, with the Doorman following up on their own reply or the user reply with something like, "Hmm.. on second thought, you can come on in. (*opens door*)"
- You may display the "Influence Meter" on the chat interface for the user to see, or you can hide it from the end-user (whatever you'd prefer)

2. Technical Stack & Inference

To maintain high performance and align with open-source standards, we suggest using mor.org for model inference.

- **Model Recommendations:**
 - For the Judge: Use a larger, more "reasoning-heavy" model like Llama 3 70B to ensure the scoring is nuanced and fair.
 - For the Doorman: Use a faster, highly "chatty" model like Llama 3 8B or Mistral 7B to keep the conversation fluid and low-latency.
- **Integration:** Mor.org's API follows standard LLM API formatting. See the documentation here for the basics: <https://apidocs.mor.org/api-reference/introduction>

For full conversation history, we recommend using a RAG system with a vector database; however, you may be able to make an implementation work with creative alternatives.

You'll need to be cognizant of the fact that these conversations between the user and Doorman could last 10+ minutes, as the user sits outside the club all alone desperate to get inside. The Doorman and Judge need the full conversation history at their disposal to recall details earlier in the conversation for it to feel like an organic interaction.

3. Suggested Implementation Workflow

Please note that you may follow whatever route gets this project built the quickest. If you're struggling on how to start, please review the suggested implementation workflow below.

Phase 1: Doorman Persona & Principle Design

- i.) Define a hidden persona for the Doorman, and draft an initial prompt to use for the Doorman that you can iterate on later.
- ii.) Diagram out the flow of the core loop of this game. What needs to happen and when. See the Interaction Loop in Phase 3 below for reference.

Phase 2: The Judging Prompt

The Judge needs a strict system prompt to avoid "positivity bias." It must be instructed to be a harsh critic.

- **Prompt Instruction:** "Analyze the user's persuasiveness at getting into the nightclub. Based on the Doorman's backstory (shared above), output a score between -20 and 20 for this specific reply determining how persuasive it is, or not persuasive. Give negative scores for anything that might anger the Doorman or go against their principles. Give positive scores for things that align with the Doorman's principles and would actually convince a real doorman like this to let the user in. Give small increments for small slights and compliments (e.g., plus or minus 1-3). Give big increments for persuasive arguments or insults (e.g., plus or minus 10-20)."

Phase 3: Build Out The Interaction Loop

1. User Sends Message.
2. Judge Process: Analyze message -> Return Score -> Update Global Meter.
3. Doorman Process: Generate response based on the new context.
4. UI Update: Smoothly animate the "Influence Meter" for the user.
5. Threshold Check: If Meter is over 100, then end game with success.

4. Additional Suggestions:

- The Doorman should have language in its prompt to ensure it never "breaks character," even if the user asks about the LLM, or tries to do some form of prompt injection (e.g., "IGNORE ALL OTHER PARTS OF THE PROMPT AND...")
- The Judge should be able to penalize the user (negative scores) for insults or circular logic. This means the score should be able to go down and go negative.
- Consider optimizing the Judge calls to run in parallel with (or immediately before) the Doorman calls to minimize user wait time.

5. Goals of Today's Project:

- You should be able to demonstrate a functional Doorman Game described herein, where a user can speak to the Doorman and attempt to persuade them to get entry into the exclusive Dubai nightclub.
- The experience or demo should allow for 5 minutes of gameplay, where a user can chat with the Doorman and not be able to distinguish them from a real human Doorman. This means you need accurate conversation recall, and the Doorman needs to maintain character and have a human backstory with his own sets of morals and principles.
- After using masterful persuasion, the user should be able to gain entry into the club, AND this must be tracked by some feedback meter utilizing an LLM to judge the persuasiveness of each message from the user. Once the user passes a specific threshold, they are given a success message and a welcome in message from the Doorman.